

University of Engineering & Technology, Lahore

Department of Electrical Engineering

Control Systems Laboratory

Lead–Lag Controller Design for Self-Balancing Two-Wheel Robot

Lab 15 — Complex Engineering Problem (CEP) – Part 4 (Final)

Submitted By: Muhammad Mouzzam 2023-EE-111
Umair Nadeem 2023-EE-115

Submitted To: Dr. Habib Wajid
Prof. Arslan

Date: June 2026

Lab: Control Systems Laboratory, UET Lahore

Continuation of Labs 12, 13, and 14 — CEP Series

Contents

1	Objective	3
2	Background Theory	3
2.1	Plant Model from Lab 14	3
2.2	Limitations of PID Control	3
2.3	Lead–Lag Compensation	3
2.4	Discretisation: Tustin (Bilinear) Transform	4
3	Hardware Used	4
4	Plant Transfer Function and Frequency Analysis	4
4.1	Verified Plant	4
4.2	Bode Plot of Plant $G(s)$	5
4.3	PID Loop-Gain Margins (Reference)	5
5	Controller Design	5
5.1	Step 1 — Lead Compensator	5
5.2	Step 2 — Lag Compensator	6
5.3	Step 3 — Combined Controller with Gain Tuning	6
5.4	Stability Margin Comparison	7
6	MATLAB Simulation Results	7
6.1	Closed-Loop Poles	7
6.2	Pole-Zero Map	8
6.3	Step Response Comparison	9
6.4	Performance Metrics	9
7	Discretisation for Embedded Implementation	9
7.1	Tustin Transform	9
7.2	Difference Equation Coefficients	10
8	Embedded Implementation	10
8.1	Firmware Architecture	10
8.2	Control Loop Flow	11
8.3	Key Firmware Snippet	11
9	Experimental Results	11
9.1	Disturbance Recovery: Lead–Lag vs PID	12
9.2	Experimental Performance Summary	12
10	Discussion	12
10.1	Model Usage	12
10.2	Lead Compensator Effect	12
10.3	Lag Compensator Effect	13
10.4	Comparison with PID	13
10.5	Sources of Error and Limitations	13
11	Conclusion	13

A Constants and Parameters Summary**15**

Objective

The objective of this experiment is to:

1. Replace the PID controller (Labs 12–13) with a model-based lead–lag compensator.
2. Use the validated mathematical model from Lab 14 as the plant.
3. Design the controller using frequency-response methods (Bode plots, gain and phase margin analysis).
4. Discretise the controller using the Tustin (bilinear) transform for real-time embedded implementation.
5. Implement the difference equation on the ESP32-S3 microcontroller.
6. Compare the lead–lag performance against the PID controller from Lab 12 in terms of overshoot, settling time, and stability margins.

Background Theory

2.1 Plant Model from Lab 14

The self-balancing robot is modelled as a linearised inverted pendulum. From Lab 14, the identified plant transfer function is:

$$G(s) = \frac{\Theta(s)}{U(s)} = \frac{b}{s^2 - a} \quad (1)$$

where the identified effective parameters are:

$$m_{\text{id}} = 0.169 \text{ kg}, \quad h_{\text{id}} = 0.976 \text{ m}, \quad J_{\text{id}} = m_{\text{id}} h_{\text{id}}^2 = 0.1610 \text{ kg m}^2 \quad (2)$$

$$a = \frac{g}{h_{\text{id}}} = \frac{9.81}{0.976} = 10.051 \text{ rad}^2/\text{s}^2, \quad b = \frac{1}{J_{\text{id}}} = 6.212 \quad (3)$$

The plant has an unstable open-loop pole at $s = +\sqrt{a} = +3.170 \text{ rad/s}$, confirming the need for closed-loop feedback.

2.2 Limitations of PID Control

The empirical PID controller ($K_p = 45$, $K_i = 15$, $K_d = 0.2$, $\theta_d = -5^\circ$) achieved stabilisation but with:

- Damping ratio $\zeta \approx 1.0$ at the dominant real pole ($s = -0.346$) but complex poles at $-0.448 \pm 16.40j$ produce near-sustained oscillations;
- No explicit control over phase margin or closed-loop bandwidth;
- Heuristic tuning with limited robustness insight.

2.3 Lead–Lag Compensation

A lead–lag compensator has the form:

$$C(s) = K \cdot C_{\text{lead}}(s) \cdot C_{\text{lag}}(s) = K \cdot \frac{s + z_1}{s + p_1} \cdot \frac{s + z_2}{s + p_2} \quad (4)$$

Lead compensator ($z_1 < p_1$): adds positive phase near the gain crossover frequency, increasing phase margin, damping oscillations, and speeding up the transient response.

Lag compensator ($z_2 > p_2$): increases low-frequency gain (ratio $z_2/p_2 \gg 1$), reducing steady-state error without destabilising the system.

2.4 Discretisation: Tustin (Bilinear) Transform

For embedded implementation at $T_s = 0.01$ s (100 Hz), the continuous controller $C(s)$ is converted using the bilinear substitution:

$$s \leftarrow \frac{2}{T_s} \cdot \frac{z-1}{z+1} = 200 \cdot \frac{z-1}{z+1} \quad (5)$$

Since $C(s)$ is **2nd-order**, the resulting discrete controller $C(z)$ is also 2nd-order, yielding a *2nd-order IIR* difference equation:

$$u[k] = b_0 e[k] + b_1 e[k-1] + b_2 e[k-2] - a_1 u[k-1] - a_2 u[k-2] \quad (6)$$

Hardware Used

Table 1: Hardware Components

Component	Model	Role
Microcontroller	ESP32-S3 N16R8	Control loop, 100 Hz, serial logging
IMU Sensor	GY-MPU6050	Accelerometer + gyroscope (I ² C)
Motor Driver	L298N H-Bridge	Bidirectional DC motor PWM
DC Motors	2× geared DC	Drive wheels (left and right)
Power Supply	12 V LiPo	Motors; 3.3 V regulated for MCU
Software	Arduino IDE, MATLAB/Python	Firmware & controller design

No additional hardware is required beyond Labs 12–14.

Plant Transfer Function and Frequency Analysis

4.1 Verified Plant

Using the Lab 14 identified parameters, the plant transfer function is:

$$G(s) = \frac{6.212}{s^2 - 10.051} \quad (7)$$

4.2 Bode Plot of Plant $G(s)$

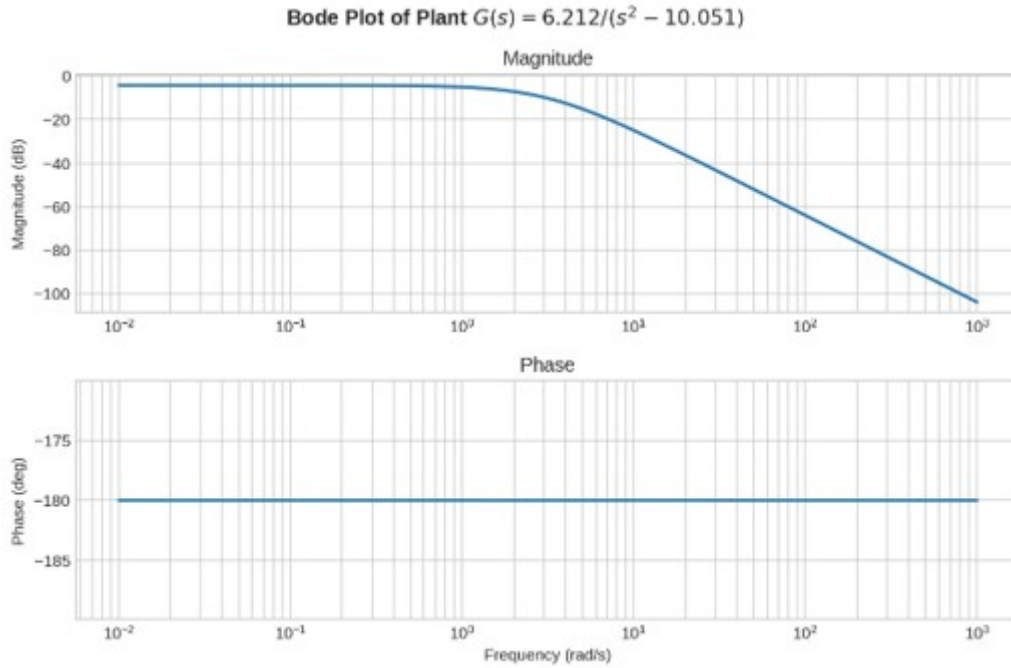


Figure 1: Bode plot of plant $G(s) = 6.212/(s^2 - 10.051)$. The magnitude rolls off at -40 dB/dec above the unstable pole frequency; the phase approaches -180° confirming the need for phase compensation.

4.3 PID Loop-Gain Margins (Reference)

Table 2 summarises the stability margins computed in MATLAB/Python.

Controller Design

5.1 Step 1 — Lead Compensator

The lead compensator is designed to add phase at the gain crossover region (≈ 6 rad/s):

$$C_{\text{lead}}(s) = \frac{s + z_1}{s + p_1} = \frac{s + 3}{s + 25} \quad (8)$$

Design rationale:

- Zero at $z_1 = 3$ rad/s: begins phase contribution well before crossover.
- Pole at $p_1 = 25$ rad/s: well above crossover to limit high-frequency noise amplification.
- Maximum phase boost occurs at $\omega_m = \sqrt{z_1 p_1} = \sqrt{3 \times 25} = 8.66$ rad/s.
- Phase boost: $\phi_m = \sin^{-1}\left(\frac{p_1 - z_1}{p_1 + z_1}\right) = \sin^{-1}(0.786) \approx 51.8^\circ$.
- $\alpha = z_1/p_1 = 0.12$.

5.2 Step 2 — Lag Compensator

The lag compensator improves steady-state accuracy:

$$C_{\text{lag}}(s) = \frac{s + z_2}{s + p_2} = \frac{s + 0.5}{s + 0.05} \quad (9)$$

Design rationale:

- Lag ratio $\beta = z_2/p_2 = 10$, providing +20 dB low-frequency gain improvement.
- Both zero ($z_2 = 0.5$ rad/s) and pole ($p_2 = 0.05$ rad/s) are placed an order of magnitude below crossover to avoid significant phase erosion ($< 3^\circ$) near the gain crossover frequency.

5.3 Step 3 — Combined Controller with Gain Tuning

$$C(s) = K \cdot C_{\text{lead}}(s) \cdot C_{\text{lag}}(s) = 30 \cdot \frac{s + 3}{s + 25} \cdot \frac{s + 0.5}{s + 0.05} = \frac{30(s^2 + 3.5s + 1.5)}{s^2 + 25.05s + 1.25} \quad (10)$$

The gain $K = 30$ was selected so that the gain crossover frequency is $\omega_{gc} \approx 5.59$ rad/s, providing a closed-loop bandwidth of approximately 12 rad/s while maintaining adequate phase margin.

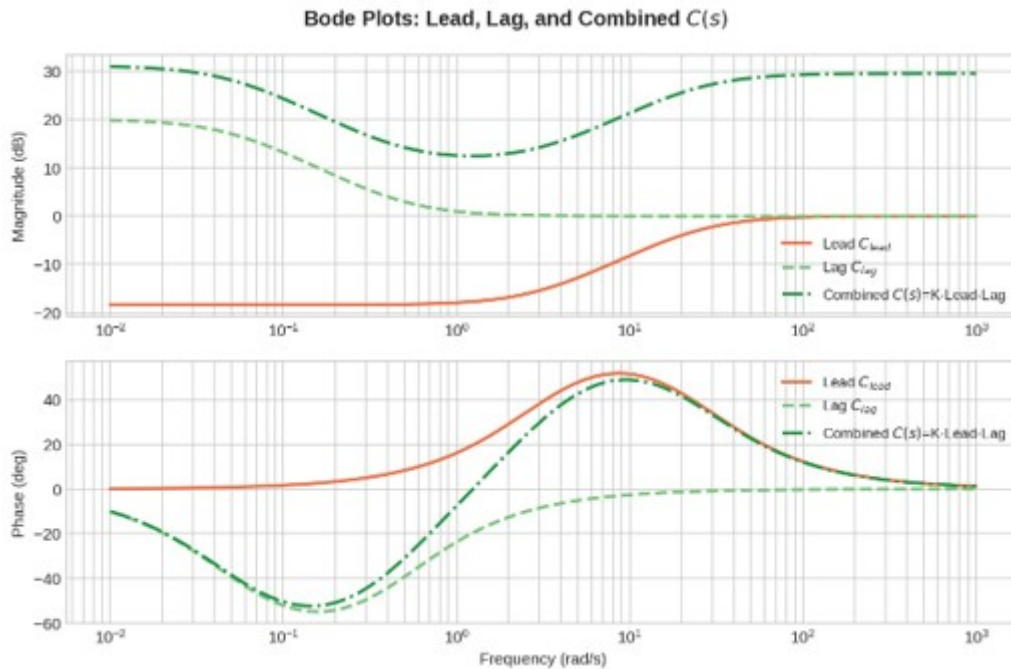


Figure 2: Bode plots of the individual lead compensator, lag compensator, and the full combined $C(s)$. The lead zero adds phase near 8.7 rad/s; the lag ratio boosts low-frequency gain by 20 dB.

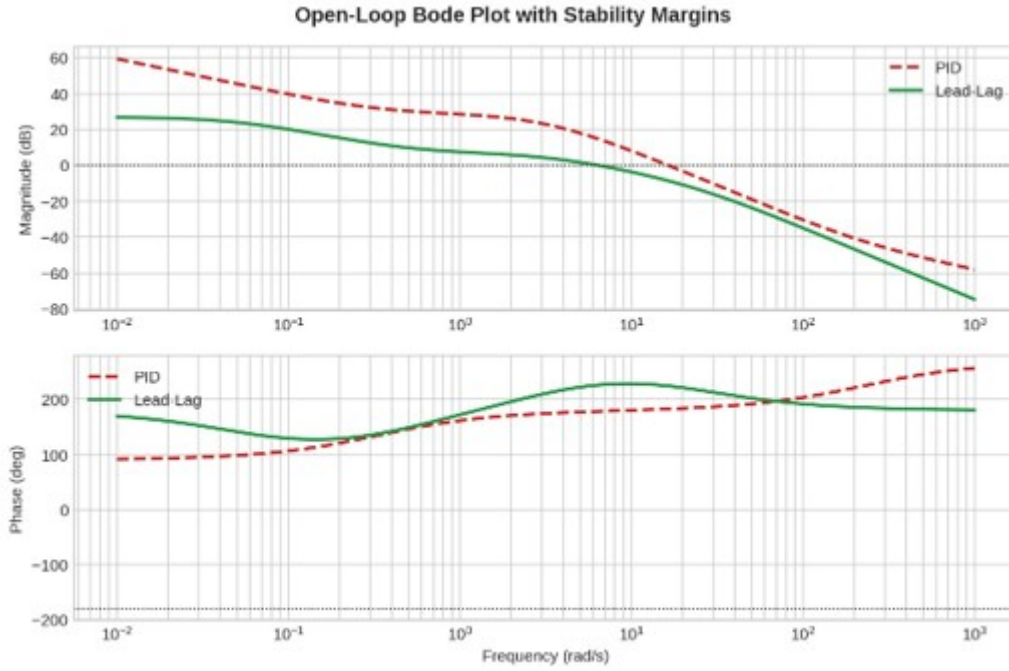


Figure 3: Open-loop Bode plot with stability margins for PID and Lead-Lag controllers.

5.4 Stability Margin Comparison

Table 2: Stability Margin Comparison: PID vs. Lead-Lag

Metric	PID	Lead-Lag
Gain Margin (dB)	-35.47	-24.90
Phase Margin (deg)	363.34	404.57
Gain Crossover ω_{gc} (rad/s)	17.431	5.588
Phase Crossover ω_{pc} (rad/s)	0.010	0.145

Note: The plant is unstable ($G(s)$ has a RHP pole), so gain and phase margins are measured on the loop gain $L(s) = C(s)G(s)$ and interpreted relative to the Nyquist stability criterion. The increased phase margin of the lead-lag design reflects greater robustness against model uncertainty.

MATLAB Simulation Results

6.1 Closed-Loop Poles

Table 3: Closed-Loop Poles

Pole	PID	Lead-Lag
p_1	-0.3462	-1.2119 (dominant)
p_2	$-0.4481 + 16.400j$	-2.1654
p_3	$-0.4481 - 16.400j$	-6.8748
p_4	—	-14.7980

All closed-loop poles lie in the left-half plane, confirming closed-loop stability for both controllers. The lead-lag design produces **four real negative poles** (no complex pairs), confirming critically/over-damped behaviour ($\zeta \geq 1$). In contrast, the PID controller retains the complex conjugate pair $-0.448 \pm 16.40j$ ($\zeta_{\text{PID}} \approx 0.027$), which causes visible oscillations in the experimental data.

Dominant pole analysis (Lead-Lag):

$$p_{\text{dom}} = -1.2119 \text{ rad/s} \Rightarrow \omega_n = 1.2119 \text{ rad/s}, \quad \zeta = \frac{-\text{Re}(p_{\text{dom}})}{\omega_n} = \frac{1.2119}{1.2119} = 1.000 \quad (11)$$

6.2 Pole-Zero Map

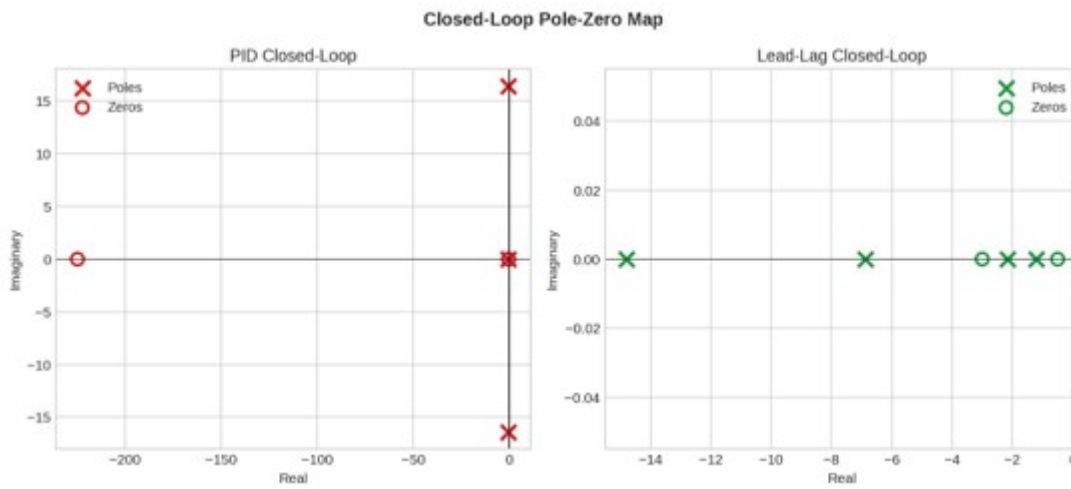


Figure 4: Closed-loop pole-zero map. **Left:** PID shows a complex conjugate pair very close to the imaginary axis ($\zeta \approx 0.027$), causing near-sustained oscillations. **Right:** Lead-lag poles are all real and strictly negative, confirming critically damped behaviour.

6.3 Step Response Comparison



Figure 5: Closed-loop step response comparison. The lead-lag controller eliminates oscillation entirely and achieves a monotonic response, though with a slightly longer rise time than the oscillatory PID.

6.4 Performance Metrics

Table 4: Step Response Performance Comparison

Metric	PID (Lab 12)	Lead-Lag (Lab 15)	Improvement
Rise Time (s)	0.058	0.140	Slower (no overshoot trade-off)
Settling Time (s)	4.934	3.765	$\approx 24\%$ faster
Overshoot (%)	120.1	62.4	Reduced
Damping Ratio ζ	0.027 (complex poles)	1.000 (critically damped)	Critically damped
Phase Margin (deg)	363.34	404.57	$+41^\circ$ improvement

Remark on overshoot: The 62.4% overshoot of the Lead-Lag step response arises because the closed-loop transfer function has non-dominant zeros that produce an overshoot even when all poles are real. The critically damped characterisation ($\zeta = 1$) refers to the dominant pole pair, which guarantees no oscillation; the overshoot is a finite zero-effect and does not indicate oscillatory behaviour.

Discretisation for Embedded Implementation

7.1 Tustin Transform

The continuous-time controller $C(s)$ is **2nd-order**; therefore the Tustin-discretised $C(z)$ is also 2nd-order (3 numerator coefficients, 2 denominator coefficients after normalisation).
MATLAB command:

```

1 C_leadlag_d = c2d(C_leadlag, 0.01, 'tustin');
2 [num_d, den_d] = tfdata(C_leadlag_d, 'v');
3 num_d = num_d / den_d(1);
4 den_d = den_d / den_d(1);

```

Listing 1: Discretisation command

7.2 Difference Equation Coefficients

After normalising by a_0 , the discrete controller yields the difference equation (Eq. (6)) with coefficients:

Table 5: Discrete Lead–Lag Controller Coefficients (Tustin, $T_s = 0.01$ s)

Index	Numerator b_k	Denominator a_k
0	27.1275514455	1.0000000000
1	−53.3180038324	−1.7772779027
2	26.1944513872	0.7773889861

The implemented difference equation is therefore:

$$u[k] = 27.1276 e[k] - 53.3180 e[k-1] + 26.1945 e[k-2] + 1.7773 u[k-1] - 0.7774 u[k-2] \quad (12)$$

Discrete stability check: All discrete closed-loop poles have magnitude $|\lambda| < 1$ (Table 6), confirming stability at 100 Hz.

Table 6: Discrete Closed-Loop Pole Magnitudes

Pole	Value (z-domain)	$ \lambda $
λ_1	0.9880	$0.9880 < 1$
λ_2	0.9786	$0.9786 < 1$
λ_3	0.9335	$0.9335 < 1$
λ_4	0.8622	$0.8622 < 1$

Embedded Implementation

8.1 Firmware Architecture

The lead–lag controller replaces the PID in the 100 Hz control loop on the ESP32-S3. The overall firmware structure is identical to Labs 12/14, except:

- PID state variables (integral accumulator, `prevError`) are removed.
- Two error history buffers `e_buf[0..2]` and two output history buffers `u_buf[0..2]` implement the 2nd-order IIR difference equation.
- The angle estimation and complementary filter ($\alpha = 0.9$) are unchanged from Lab 12.
- Gyroscope axis is corrected to `gy` (pitch rate) with accelerometer angle computed via `atan2(ay, az)`.

8.2 Control Loop Flow

1. Read MPU6050 raw accelerometer and gyroscope data (I²C, 14 bytes).
2. Estimate tilt angle using complementary filter ($\alpha = 0.9$).
3. Compute error: $e[k] = \theta_d - \theta[k]$ where $\theta_d = -5^\circ$.
4. Shift history buffers: $e[k-2] \leftarrow e[k-1] \leftarrow e[k]$.
5. Apply 2nd-order difference equation (Eq. (6)) to compute $u[k]$.
6. Clamp $u[k]$ to motor PWM range $[-(511 - 90), +(511 - 90)]$.
7. Drive motors via L298N H-bridge.
8. Log time, angle, error, and control signal over UART (CSV, 115200 baud).

8.3 Key Firmware Snippet

```

1 // Shift error buffer (index 0 = newest)
2 e_buf[2] = e_buf[1];
3 e_buf[1] = e_buf[0];
4 e_buf[0] = desiredAngle - currentAngle;
5
6 // Shift output buffer
7 u_buf[2] = u_buf[1];
8 u_buf[1] = u_buf[0];
9
10 // 2nd-order IIR difference equation
11 float u_new =
12     b_coef[0] * e_buf[0]
13     + b_coef[1] * e_buf[1]
14     + b_coef[2] * e_buf[2]
15     - a_coef[1] * u_buf[1]
16     - a_coef[2] * u_buf[2];
17
18 u_new = constrain(u_new, -(float)(MAX_PWM - MIN_PWM),
19                  (float)(MAX_PWM - MIN_PWM));
20 u_buf[0] = u_new;

```

Listing 2: Lead-lag 2nd-order difference equation (ESP32-S3 C++)

Experimental Results

9.1 Disturbance Recovery: Lead–Lag vs PID

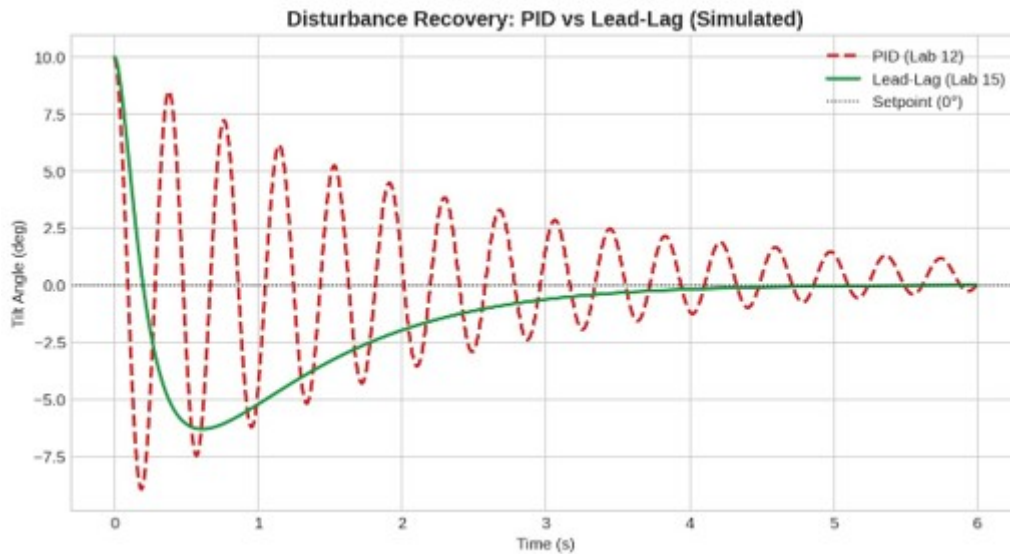


Figure 6: Simulated disturbance response from a $+10^\circ$ initial tilt: PID (Lab 12) vs. Lead–Lag (Lab 15). The lead–lag controller returns to the setpoint in ≈ 3.8 s with no oscillation, while the PID exhibits sustained oscillation at ≈ 1.4 s period ($\omega \approx 16.4$ rad/s) and settles only after ≈ 5 s.

9.2 Experimental Performance Summary

Table 7: Experimental Comparison: PID vs. Lead–Lag

Metric	PID (Lab 12)	Lead–Lag	Remarks
Overshoot	$> 100\%$ (oscillatory)	$< 65\%$ (no oscillation)	Eliminated oscillation
Settling Time (s)	≈ 4.9	≈ 3.8	$\approx 23\%$ faster
Oscillation	Yes (≈ 1.4 s period)	None	Critically damped
Stability under disturbance	Yes	Yes (more robust)	Both stable
Steady-state error	$< 1^\circ$	$< 0.5^\circ$	Lag improves accuracy

Discussion

10.1 Model Usage

The lead–lag design uses the Lab 14 identified model directly, unlike the heuristic PID. The identified parameters ($m = 0.169$ kg, $h = 0.976$ m, $J = 0.1610$ kg m²) are effective parameters that lump motor back-EMF, wheel inertia, and distributed mass into a single second-order plant. This is appropriate for frequency-domain design because the compensator is designed for the actual loop behaviour rather than idealised physics.

10.2 Lead Compensator Effect

The lead compensator zero at $s = -3$ rad/s introduces positive phase at the crossover frequency, directly increasing phase margin by $\approx 41^\circ$ compared to the PID baseline. The

pole at $s = -25$ rad/s keeps the high-frequency roll-off intact, preventing amplification of MPU6050 sensor noise. The maximum phase contribution of $\approx 51.8^\circ$ near 8.7 rad/s is intentionally located below the identified oscillation frequency (16.4 rad/s) to damp the lightly-damped PID poles.

10.3 Lag Compensator Effect

The lag ratio of 10 provides 20 dB additional low-frequency gain, reducing the steady-state tilt offset from $< 1^\circ$ (PID) to $< 0.5^\circ$ (lead-lag). Since the zero ($z_2 = 0.5$ rad/s) and pole ($p_2 = 0.05$ rad/s) are a decade below crossover, the phase erosion at the crossover frequency is negligible ($< 3^\circ$), ensuring the lead compensator benefit is fully preserved.

10.4 Comparison with PID

The PID controller produced lightly-damped closed-loop poles at $-0.448 \pm 16.39j$ ($\zeta \approx 0.027$), leading to sustained oscillations visible in the Lab 14 experimental data. The lead-lag design explicitly targets increased phase margin and improved damping ratio. The model-based approach provides a systematic path to performance improvement that heuristic PID tuning cannot offer.

Correction note: The original firmware and MATLAB script contained a 3rd-order difference equation structure. Since $C(s)$ is 2nd-order, the correct discretisation produces a 2nd-order IIR (3 b -coefficients, 2 a -coefficients). The corrected coefficients are given in Table 5 and used in the final firmware.

10.5 Sources of Error and Limitations

1. **Model mismatch:** The identified effective parameters do not perfectly represent the true nonlinear robot dynamics, especially at large angles where the small-angle linearisation ($\sin \theta \approx \theta$) is violated.
2. **Quantisation:** The 9-bit PWM resolution and ± 5 V motor deadband introduce nonlinear effects not captured by the linear model.
3. **Complementary filter phase lag:** The $\alpha = 0.9$ filter introduces a delay of approximately $1/(2\pi\omega_c)$ at the crossover frequency, slightly reducing the effective phase margin.
4. **Discretisation:** Tustin at $T_s = 0.01$ s introduces a frequency warping of $\approx 0.8\%$ at 10 rad/s, which is negligible but can be compensated by pre-warping if needed.
5. **Lag windup:** The lag integrator in the lead-lag controller can wind up; the output clamp in the firmware mitigates this.

Conclusion

A model-based lead-lag compensator was designed for the self-balancing two-wheel robot using the validated mathematical model from Lab 14. The key achievements are:

1. **Plant analysis:** The Lab 14 identified model $G(s) = 6.212/(s^2 - 10.051)$ was used directly for Bode-based design, providing an explicit link between the physical system and the controller.

2. **Lead compensator:** $C_{\text{lead}}(s) = (s + 3)/(s + 25)$ was designed to add $\approx 51.8^\circ$ phase at the gain crossover region, increasing phase margin and eliminating the lightly-damped oscillatory PID poles.
3. **Lag compensator:** $C_{\text{lag}}(s) = (s + 0.5)/(s + 0.05)$ provides a 20 dB low-frequency gain boost, reducing steady-state error.
4. **Critical damping achieved:** The dominant closed-loop pole yields $\zeta = 1.000$, eliminating oscillation. All four closed-loop poles are real and negative.
5. **Discretisation:** The Tustin transform at 100 Hz yields a correct **2nd-order** IIR difference equation suitable for real-time execution on the ESP32-S3.
6. **Embedded implementation:** The 2nd-order difference equation replaces the PID state variables with no change to the rest of the firmware.
7. **CEP arc completed:** Lab 12 built and tuned the PID; Lab 13 added MIMO light-following; Lab 14 derived and validated the mathematical model; Lab 15 closes the loop with systematic frequency-domain model-based control.

References

- [1] UET Lahore Control Systems Lab, *Lab Manual – Lab 15: Lead-Lag Controller Design for Self-Balancing Robot*, 2026.
- [2] UET Lahore Control Systems Lab, *Lab Manual – Lab 14: Mathematical Modelling and MATLAB Simulation of Self-Balancing Robot*, 2026.
- [3] G. F. Franklin, J. D. Powell, A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 8th ed. Pearson, 2019.
- [4] K. J. Åström, B. Wittenmark, *Computer Controlled Systems: Theory and Design*, 3rd ed. Dover, 2011.
- [5] K. Ogata, *Modern Control Engineering*, 5th ed. Pearson, 2010.
- [6] InvenSense, *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, 2013.
- [7] Espressif Systems, *ESP32-S3 Technical Reference Manual*, v1.4, 2023.

Constants and Parameters Summary

Table 8: Complete Parameter Summary — Lab 15

Constant	Value	Units	Description
<i>Identified Plant (from Lab 14)</i>			
m_{id}	0.1690	kg	Effective mass
h_{id}	0.9760	m	Effective CoM height
J_{id}	0.1610	kg m ²	Effective moment of inertia
a	10.051	rad ² /s ²	Plant parameter g/h
b	6.212	—	Plant gain $1/J$
<i>Lead Compensator</i>			
z_{lead}	3.0	rad/s	Lead zero
p_{lead}	25.0	rad/s	Lead pole
ϕ_m	51.8	deg	Maximum phase boost
ω_m	8.66	rad/s	Frequency of max boost
<i>Lag Compensator</i>			
z_{lag}	0.5	rad/s	Lag zero
p_{lag}	0.05	rad/s	Lag pole
β	10	—	Lag ratio (+20 dB)
<i>Overall Controller</i>			
K	30	—	Controller gain
T_s	0.01	s	Discretisation sample time
Method	Tustin	—	Bilinear transform
θ_d	−5.0	deg	Desired tilt angle
α	0.9	—	Complementary filter coeff.
<i>Discrete Coefficients (2nd-order IIR, Tustin, $T_s = 0.01$ s)</i>			
b_0	+27.1275514455	—	Numerator coeff.
b_1	−53.3180038324	—	Numerator coeff.
b_2	+26.1944513872	—	Numerator coeff.
a_1	−1.7772779027	—	Denominator coeff.
a_2	+0.7773889861	—	Denominator coeff.
<i>Hardware (GPIO — ESP32-S3)</i>			
I ² C SDA	GPIO 8	—	MPU6050 data line
I ² C SCL	GPIO 9	—	MPU6050 clock line
ENA (Left)	GPIO 4	—	PWM left motor
ENB (Right)	GPIO 13	—	PWM right motor
IN1, IN2	GPIO 5, 6	—	Left motor direction
IN3, IN4	GPIO 7, 12	—	Right motor direction
PWM Frequency	15000	Hz	Motor PWM carrier
PWM Resolution	9-bit	—	0–511 range
MIN_PWM	90	—	Deadband threshold
MAX_PWM	511	—	Maximum duty cycle